

Аппаратно-Программное обеспечение ЭВМ и Сетей» часть 2 (Технологии локальных сетей)

Садовский В.Т.

ст. преподаватель кафедры АСУ БРУ

Тема 22 Транспортные протоколы. Протокол ТСР.

Протокол ТСР применяется в тех случаях, когда требуется гарантированная доставка сообщений. Он использует контрольные суммы пакетов для проверки их целостности и освобождает прикладные процессы от необходимости таймаутов и повторных передач для обеспечения надежности. Для отслеживания подтверждения доставки в ТСР реализуется алгоритм "скользящего" окна. Наиболее типичными прикладными процессами, использующими ТСР, являются FTP (File Transfer Protocol - протокол передачи файлов), telnet, системы электронной почты SMTP, HTTP, X-window, RCP (remote copy) и др.

Информация, поступающая к протоколу ТСР от протоколов более высокого уровня, поступает непрерывным потоком. Для протокола ТСР – это *неструктурированный поток байтов*.

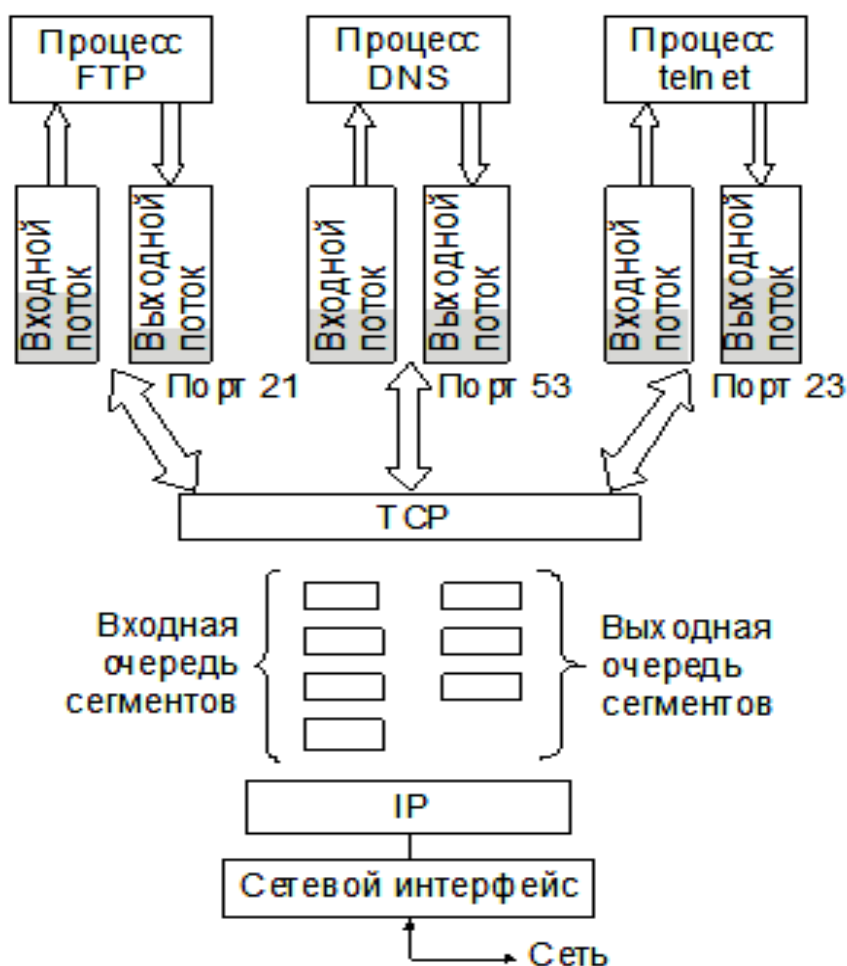


Рисунок 20.1.
Формирование ТСР-
сегментов из потока байтов

Протокол ТСР с пользовательской информацией поступает следующим образом:

1. поступающие данные буферизует;

2. далее из буфера «нарезаются» части (блоки) информации называемые сегментами (Рисунок 21.2)
3. сегмент снабжается заголовком (Рисунок 21.1) и передается на сетевой уровень.

(Сегментом называют как единицу передаваемых данных в целом (поле данных и заголовок TCP), так и отдельно поле данных.)

Заголовок TCP-сегмента содержит значительно больше полей, чем заголовок UDP, что отражает более развитые возможности протокола:

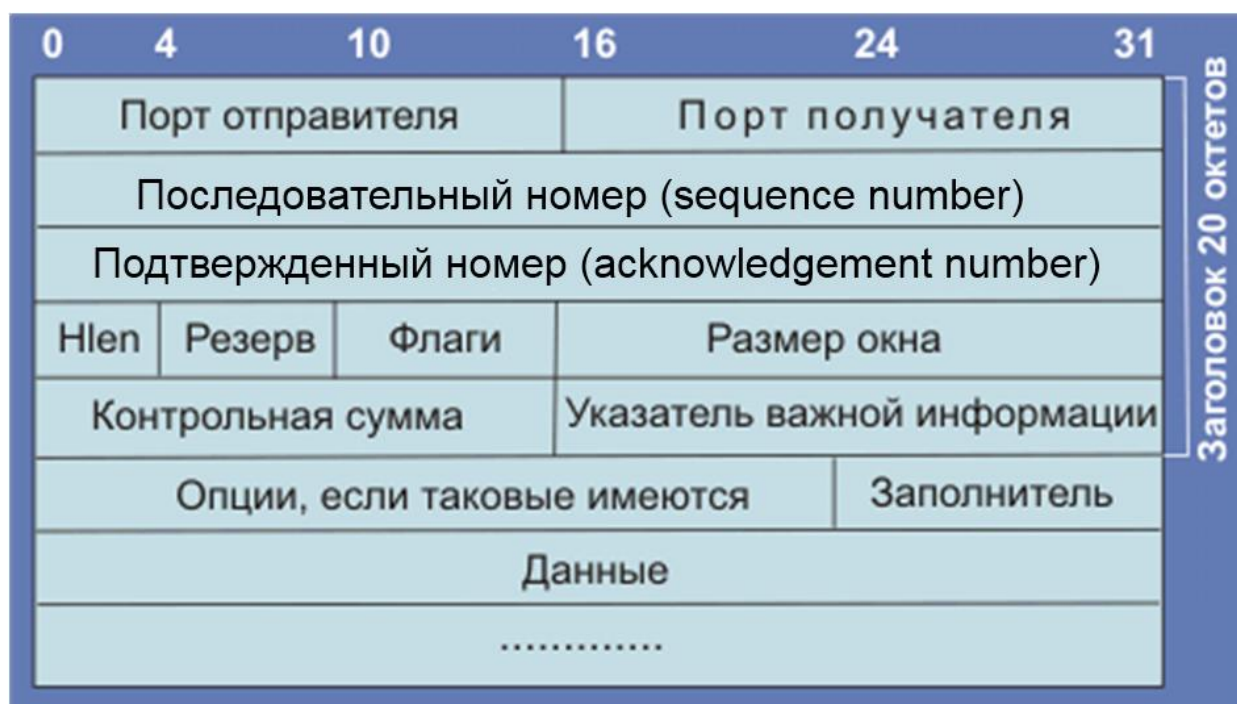


Рисунок 20.2. Заголовок сегмента транспортного протокола TCP

Поля заголовка TCP-сегмента

1. **Порт источника (source port)** занимает 2 байта и идентифицирует процесс-отправитель.
2. **Порт приемника (destination port)** занимает 2 байта и идентифицирует процесс-получатель.
3. **Последовательный номер (sequence number)** занимает 4 байта и представляет собой номер байта, который определяет смещение сегмента относительно потока отправляемых данных (другими словами, номер первого байта данных в сегменте).
4. **Подтвержденный номер (acknowledgement number)** занимает 4 байта и содержит максимальный номер байта в полученном сегменте, увеличенный на единицу. Именно это значение используется в качестве квитанции. Если установлен контрольный бит ACK, то это поле содержит следующий номер очереди, который отправитель данного сегмента желает получить в обратном направлении.
5. **Длина заголовка (hlen)** занимает 4 бита и представляет собой длину заголовка TCP-сегмента, измеренную в 32-битовых словах. Длина заголовка

не фиксирована и может изменяться в зависимости от значений, устанавливаемых в поле параметров.

6. **Резерв (reserved)** занимает 6 бит.
7. **Флаги Управляющие биты (code bits)** числом 6 содержат служебную информацию о типе данного сегмента. Положительное значение сигнализируется установкой этих битов в единицу:
 1. **URG** — срочное сообщение;
 2. **ACK** — квитанция на принятый сегмент;
 3. **PSH** — запрос на отправку сообщения без ожидания заполнения буфера (протокол TCP может выжидать заполнения буфера перед отправкой сегмента, но если требуется срочная передача, то приложение сообщает об этом протоколу TCP с помощью данного бита);
 4. **RST** — запрос на восстановление соединения;
 5. **SYN** — сообщение, используемое для синхронизации счетчиков переданных данных при установлении соединения;
 6. **FIN** — признак достижения передающей стороной последнего байта в потоке передаваемых данных.
8. **Окно (window)** занимает 2 байта и задает количество байтов данных, ожидаемых отправителем данного сегмента, начиная с байта, номер которого указан в поле подтвержденного номера.
9. Контрольная сумма (checksum) занимает 2 байта.
10. **Указатель срочности (urgent pointer)** занимает 2 байта и указывает на конец данных, которые необходимо срочно принять, несмотря на переполнение буфера. Указатель срочности используется совместно с кодовым битом URG. То есть если какие-то данные необходимо переслать приложению-получателю вне очереди, то приложение-отправитель должно сообщить об этом протоколу TCP путем установки в единицу бита URG.
11. **Параметры (options)** имеют переменную длину и могут вообще отсутствовать. Максимальная величина поля составляет 3 байта; оно используется для решения вспомогательных задач, например для выбора максимального размера сегмента. Поле параметров может располагаться в конце заголовка TCP, а его длина кратна 8 бит.
12. **Заполнитель (padding)** может иметь переменную длину. Это фиктивное поле, используемое для доведения размера заголовка до целого числа 32-битовых слов.

Основным отличием TCP от UDP является то, что на протокол TCP возложена дополнительная задача — обеспечить надёжную доставку сообщений, используя в качестве основы ненадёжный дейтаграммный протокол IP.

На Рисунке 20.3 показаны сети, соединенные маршрутизаторами, на которых установлен протокол IP. Установленные **на конечных узлах** протокольные модули TCP решают задачу обеспечения надежного обмена данными путем установления между собой логических соединений. Благодаря логическому соединению TCP следит, чтобы передаваемые сегменты не были потеряны, не были

продублированы и пришли к получателю в том порядке, в котором были отправлены.

При установлении логического соединения модули TCP договариваются между собой о параметрах процедуры обмена данными. В протоколе TCP каждая сторона соединения посылает противоположной стороне следующие параметры:

- ✓ **максимальный размер сегмента**, который она готова принимать;
- ✓ **максимальный объем данных** (возможно несколько сегментов - **размер окна**); которые она разрешает другой стороне передавать в свою сторону, даже если та еще не получила квитанцию на предыдущую порцию данных
- ✓ **начальный порядковый номер байта**, с которого она начинает отсчет потока данных в рамках данного соединения.

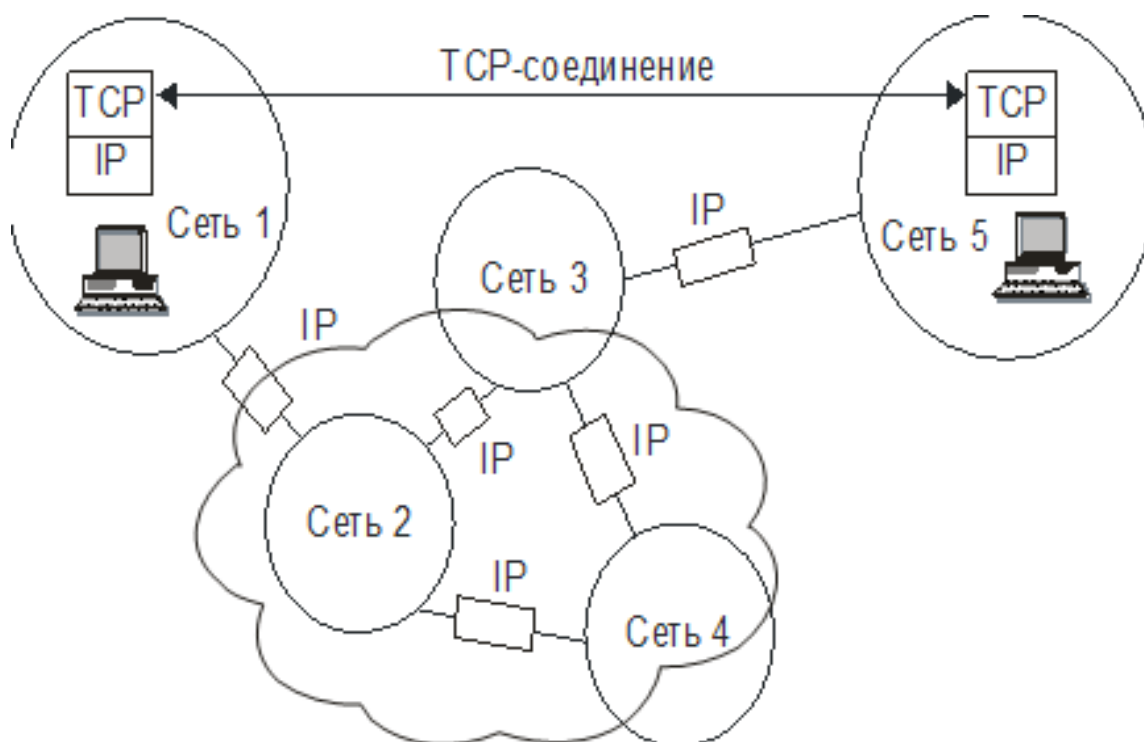


Рисунок 20.3. TCP-соединение создает надежный логический канал между конечными узлами

В результате переговорного процесса модулей TCP с двух сторон соединения определяются параметры соединения. Одни из них остаются постоянными в течение всего сеанса связи, а другие адаптивно изменяются. В частности, в зависимости от загрузки буфера принимающей стороны, а также надежности работы сети **динамически изменяется размер окна отправителя**.

Создание соединения означает также выделение операционной системой на каждой стороне соединения определенных системных ресурсов: для организации буферов, таймеров, счетчиков. Эти ресурсы будут закреплены за соединением с момента создания и до момента разрыва.

Логическое соединение TCP однозначно идентифицируется парой сокетов.

Каждый сокет одновременно может участвовать в нескольких соединениях. Так, если $(IP1, n1)$, $(IP2, n2)$, $(IP3, n3)$ — сокеты трех разных приложений, где $IP1$,

IP2, IP3 — их IP-адреса, а n1, n2, n3 — номера их TCP-портов, то возможно образование следующих соединений:

- ✓ соединение 1 - {(IP2, n2), (IP1, n1)};
- ✓ соединение 2 - {(IP1, n1), (IP3, n3)};
- ✓ соединение 3 - {(IP2, n2), (IP3, n3)}.

На Рисунке 20.3 соединения 1 и 3, образованные сокетом (IP2, n2).

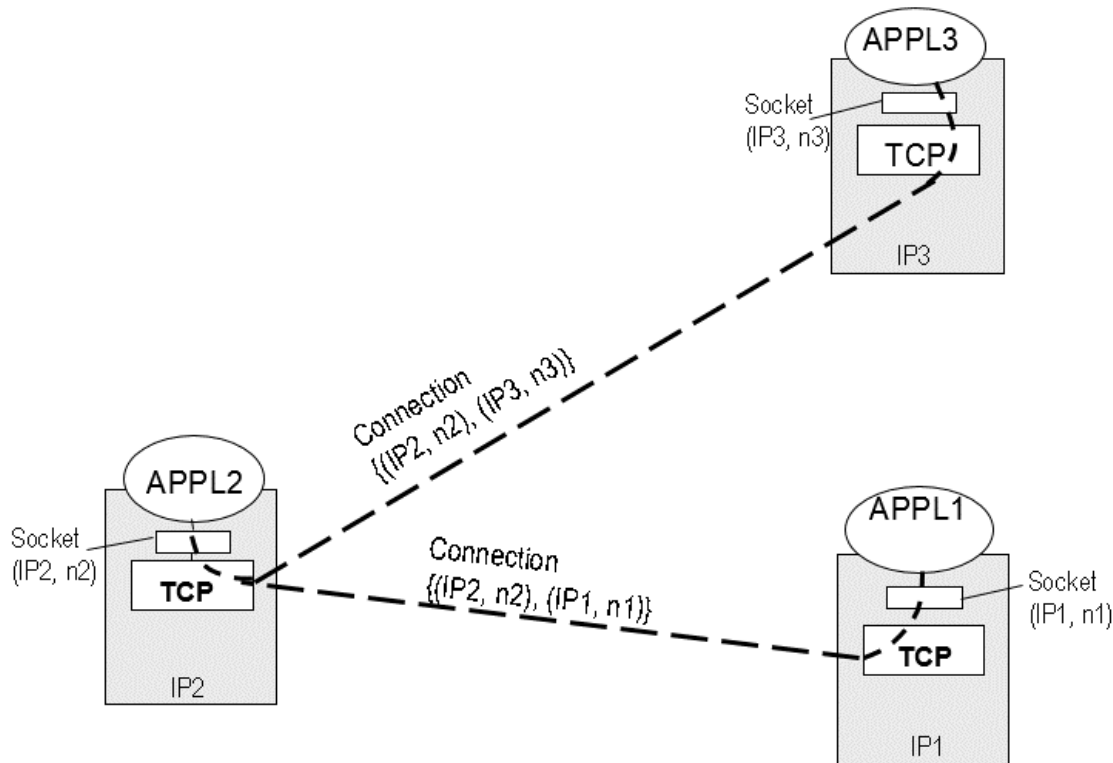


Рисунок 20.4. Один сокет может участвовать в нескольких соединениях

Рассмотрим на примере, как протокол TCP выполняет демультиплексирование. Пусть некий поставщик услуг оказывает услугу по веб-хостингу, то есть на его компьютере клиенты могут устанавливать свои веб-серверы. Веб-сервер основан на протоколе прикладного уровня HTTP, который, в свою очередь, использует протокол TCP. TCP ожидает запросы от веб-клиентов (браузеров), прослушивая хорошо известный порт 80.

На Рисунок 21.5 показан вариант хостинга с двумя веб-серверами — сервером **www1.model.ru**, имеющим IP-адрес IP1, и сервером **www2.tour.ru** с адресом IP2. К каждому из них может обращаться множество клиентов, причем клиенты могут одновременно работать как с сервером WWW1, так и с сервером WWW2. Работа каждого клиента требует сохранения прочитанных страниц, параметров и настроек сеанса связи, то есть *образования отдельного логического соединения*. Такое соединение создается протоколом TCP для каждой пары клиент-сервер.

На рисунке показаны два браузера, имеющие соответственно сокеты (IPk, nk) и (IPm, pm). Пользователь браузера обращается одновременно к серверам WWW1 и WWW2. Наличие отдельных соединений для работы с каждым из этих серверов гарантирует разделение информационных потоков — у пользователя никогда не

возникает вопроса, каким сервером ему была послана та или иная страница. Одновременно с пользователем браузера **k** с сервером WWW2 работает пользователь браузера **m**. И в этом случае отдельные логические соединения, в рамках которых идет работа обоих пользователей, позволяют изолировать их информационные потоки. На рисунке показаны буферы, количество которых определяется не числом веб-серверов и не числом клиентов, а числом логических соединений. Сообщения в эти буферы направляются в зависимости от значений сокетов как отправителя, так и получателя.

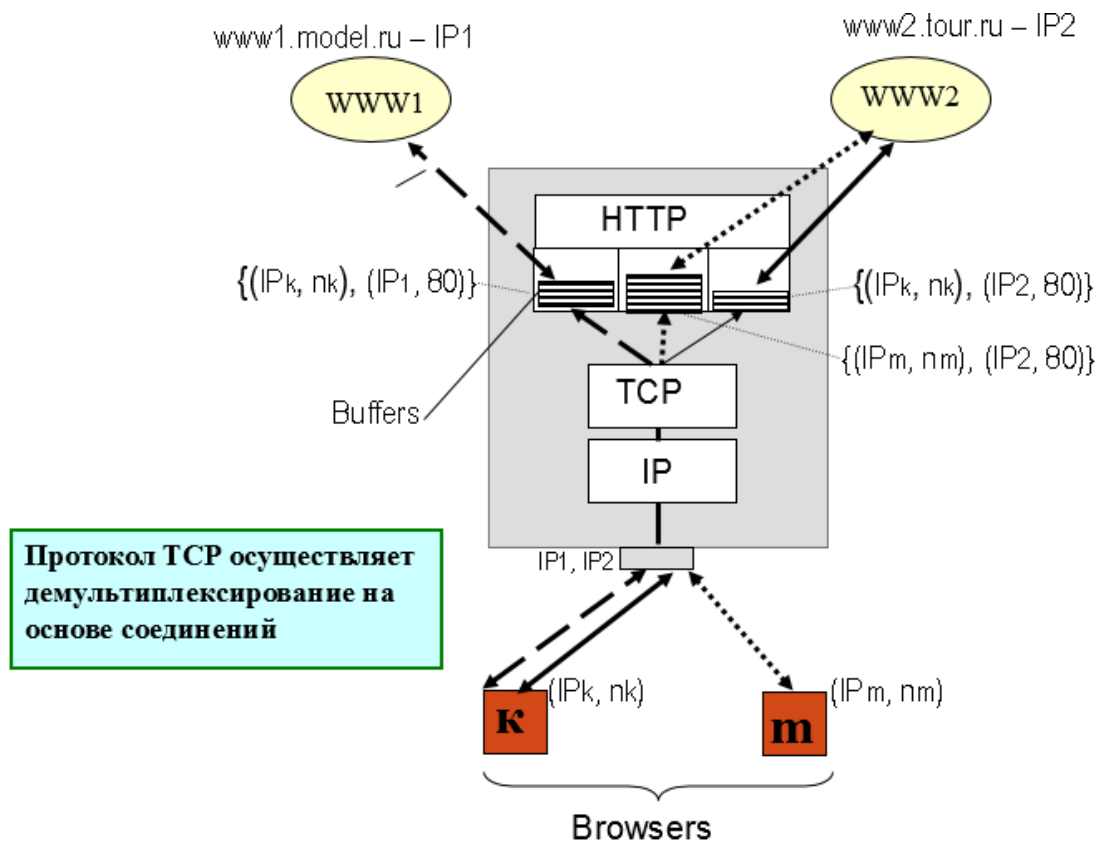


Рисунок 21.5. . Демультиплексирование TCP на основе соединений

Последовательный и подтвержденный номер

В рамках установленного соединения в протоколе TCP правильность передачи каждого сегмента должна подтверждаться квитанцией от получателя. **Квитирование** — это один из традиционных методов обеспечения надежной связи. В протоколе TCP используется частный случай квитирования — алгоритм скользящего окна.

Алгоритм скользящего окна в протоколе TCP имеет особенность. Хотя единицей передаваемых данных является сегмент, окно определено на множество нумерованных байтов неструктурированного потока данных, поступающего с верхнего уровня и буферизуемых протоколом TCP.

При установлении соединения обе стороны договариваются о начальном номере байта, с которого будет вестись отсчет в течение всего данного соединения. У каждой стороны свой начальный номер. Идентификатором каждого сегмента является номер его первого байта. Нумерация байтов в пределах сегмента

осуществляется так, что первый байт данных сразу вслед за заголовком имеет наименьший номер, а следующие за ним байты имеют следующие порядковые номера (Рисунок 21.6).

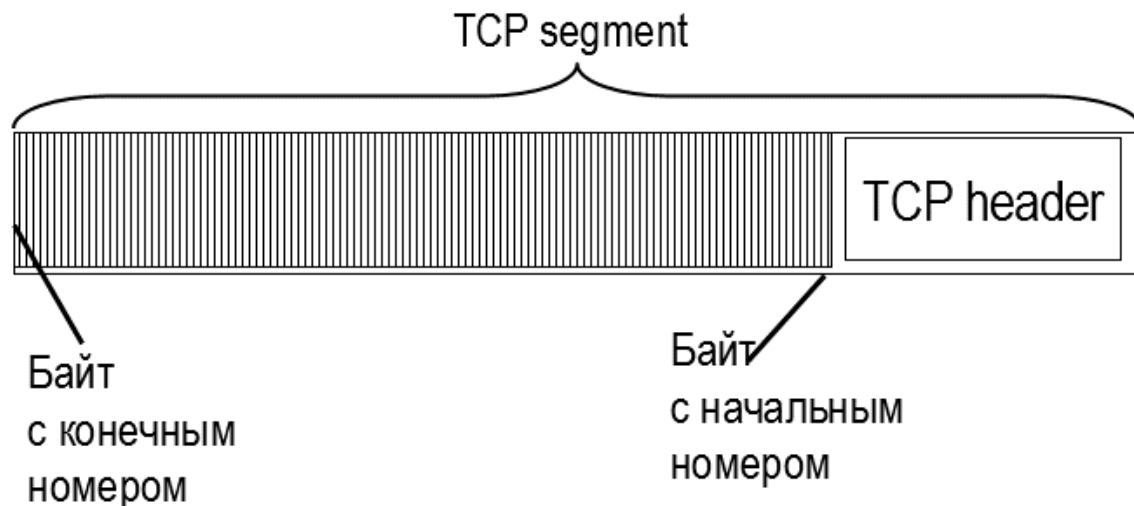


Рисунок 21.6. Нумерация байтов в TCP-сегменте

Когда отправитель посылает TCP-сегмент, он в качестве идентификатора сегмента помещает в поле последовательного номера номер первого байта данного сегмента. Так, на Рисунке 21.7 идентификаторами сегментов являются номера 32600, 34060, 35520 и т. д. На основании этих номеров TCP-получатель не только отличает данный сегмент от других, но и позиционирует полученный фрагмент относительно общего потока байтов. Кроме того, он может сделать вывод, что полученный сегмент является дубликатом или что между двумя полученными сегментами пропущены данные и т. д.

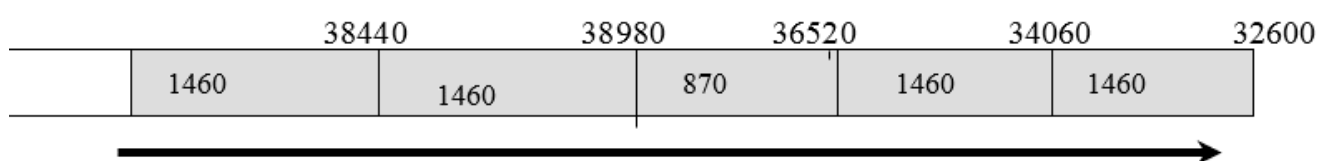


Рисунок 21.7 Порядковый номер и номер квитанции

В качестве квитанции получатель сегмента отправляет ответное сообщение (сегмент), в которое помещает число (подтверждающий номер), на единицу превышающее максимальный номер байта в полученном сегменте. Для сегментов, изображенных на рис. 5-25.7, квитанцией о получении (подтвержденным номером) служат номера последнего байта каждого сегмента +1. Так для первого отправленного сегмента это будет число $34060+1$, для второго — $35520+1$ и т. д. Подтверждающий номер часто интерпретируют как номер следующего ожидаемого байта данных. Квитанция (подтверждение) в протоколе TCP посылается только в случае правильного приема данных, отрицательные квитанции

не посылаются. Таким образом, отсутствие квитанции означает либо потерю сегмента, либо прием искаженного сегмента, либо потерю квитанции.

В протоколе ТСР в одном и том же сегменте могут быть помещены и данные, которые посылает приложение другой стороне, и квитанция, которой модуль ТСР подтверждает получение данных.

Окно приема

Протокол ТСР является дуплексным, то есть в рамках одного соединения регламентируется процедура обмена данными в обе стороны.

Каждая сторона одновременно выступает и как отправитель, и как получатель.

У каждой стороны есть пара буферов: один — для хранения принятых сегментов, другой — для сегментов, которые только еще предстоит отправить. Кроме того, имеется буфер для хранения копий сегментов, которые были отправлены, но квитанции о получении которых еще не поступили (Рисунок 21.8).

И при установлении соединения, и в ходе передачи обе стороны, выступая в роли получателя, посылают друг другу так называемые **окна приема**. Каждая из сторон, получив окно приема, «понимает», сколько байтов ей разрешается отправить с момента получения последней квитанции. Другими словами, посылая окна приема, обе стороны пытаются регулировать поток байтов в свою сторону, сообщая своему «визави», какое количество байтов (начиная с номера байта, о котором уже была выслана квитанция) они готовы в настоящий момент принять.

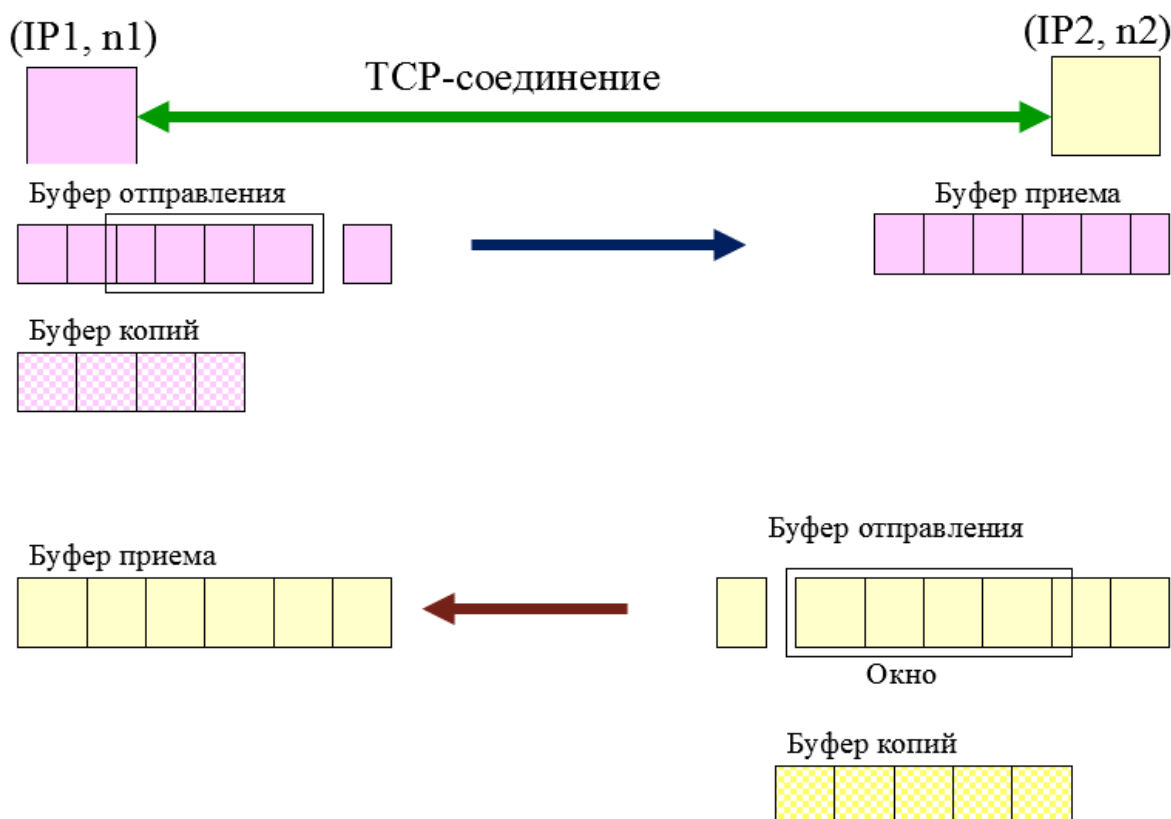


Рисунок 21.8. Система буферов ТСР-соединения

На Рисунок 21.9 показан поток байтов, поступающий с верхнего уровня в выходной буфер протокола TCP. Из потока байтов модуль TCP «нарезает» последовательность сегментов и готовит их для отправки другому сокету. Для определенности на рисунке принято направление перемещения данных справа налево.

В этом потоке можно указать несколько **логических границ**. Первая граница отделяет сегменты, которые уже были отправлены и на которые уже пришли квитанции. По другую сторону этой границы располагается окно размером W байт. Часть байтов, входящих в окно, составляют сегменты, которые также уже отправлены, но квитанции на них пока не получены. Оставшаяся часть окна — это сегменты, которые пока не отправлены, но могут быть отправлены, так как входят в пределы окна. И наконец, последняя граница указывает на начало последовательности сегментов, ни один из которых не может быть отправлен до тех пор, пока не придет очередная квитанция и окно не будет сдвинуто вправо.

Если размер окна равен W , а последняя по времени квитанция содержала значение N , то отправитель может посылать новые сегменты до тех пор, пока в очередной сегмент не попадет байт с номером $N + W$. Этот сегмент выходит за рамки окна, и передачу в таком случае необходимо приостановить до прихода следующей квитанции.

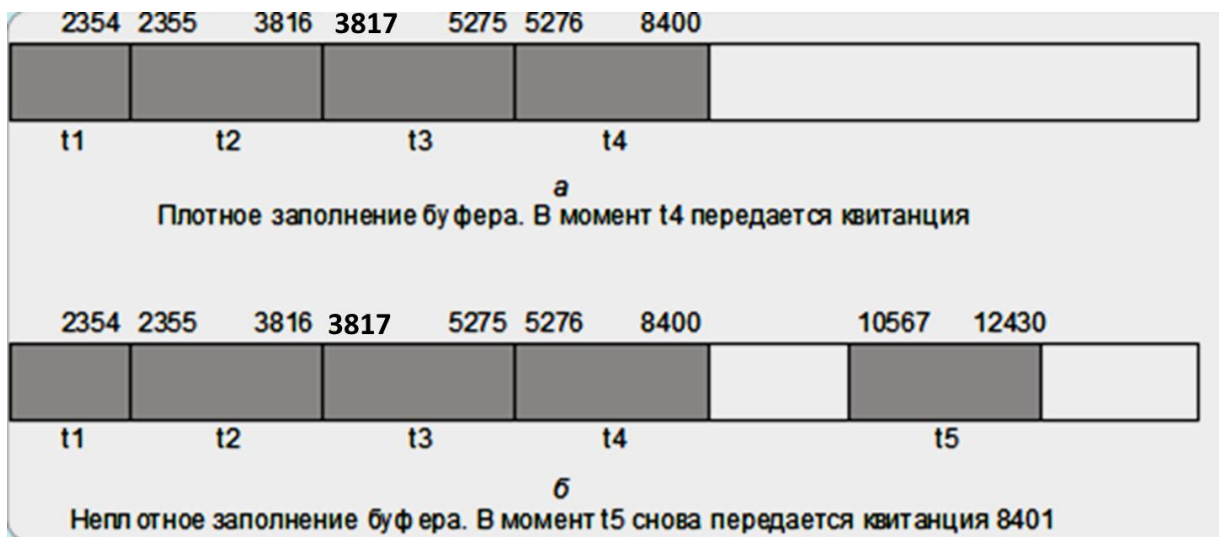


Рисунок 21.9. Особенности реализации алгоритма скользящего окна в протоколе TCP

Накопительный принцип квитирования

Получатель может послать квитанцию, подтверждающую получение сразу нескольких сегментов, если они образуют непрерывный поток байтов. Например (Рисунок 21.10,а), если в буфер, плотно без пропусков заполненный потоком байтов до 2354 включительно, поочередно поступили сегменты (2355-3816), (3817-5275) и (5276-8400), где цифры в скобках означают номера первых и последних байтов каждого сегмента, то получателю достаточно отправить только одну квитанцию на все три сегмента, указав в ней в качестве номера квитанции значение 8401. Таким образом, процесс квитирования является накопительным. Однако сегменты могут прийти к получателю не в том порядке, в котором были посланы, то есть в приемном буфере может образоваться «прогалина» (Рисунок 21.10, б).

Пусть, к примеру, после указанных выше трех сегментов вместо следующего по порядку сегмента (8401-10566) пришел сегмент (10567-12430). Очевидно, что послать в качестве номера квитанции значение 12431 нельзя, потому что это бы означало, что получены все байты вплоть до 12430. Поскольку в потоке байтов образовался разрыв, получатель может только еще раз повторить квитанцию 8401, говоря тем самым, что все еще ожидает поступления потока байтов, начиная с 8401.



Из этого примера видно, что, в отличие от многих других протоколов, протокол ТСП подтверждает получение не отдельных блоков данных, а непрерывной последовательности байтов.

Рисунок 21.10. Накопительный принцип квитирования

Время ожидания квитанции

Когда протокол ТСП передает в сеть сегмент, он «на всякий случай» помещает его копию в очередь повторной передачи и запускает таймер. Когда приходит квитанция на этот сегмент, соответствующая копия удаляется из очереди. Если же квитанция не приходит до истечения срока, то сегмент посылается повторно. Может случиться так, что повторный сегмент придет тогда, когда исходный сегмент уже окажется на месте, тогда дубликат будет попросту отброшен.

Каким должно быть время ожидания (тайм-аут) очередной квитанции? От решения этой задачи зависит производительность протокола ТСП. Тайм-аут не должен быть слишком коротким, чтобы по возможности исключить избыточные повторные передачи, снижающие полезную пропускную способность системы, но он не должен быть и слишком длинным, чтобы избежать длительных простоев, связанных с ожиданием несуществующей или «заблудившейся» квитанции.

При выборе величины тайм-аута должны учитываться скорость и надежность линий связи, их протяженность и многие другие факторы. **В протоколе ТСП тайм-аут определяется с помощью достаточно сложного адаптивного алгоритма, идея которого состоит в следующем.**

При каждой передаче засекается время от момента отправки сегмента до прихода квитанции о его приеме (время оборота). Получаемые значения времени оборота усредняются с весовыми коэффициентами, возрастающими от предыдущего замера к последующему. Это делается с тем, чтобы усилить влияние последних замеров. **В качестве тайм-аута выбирается среднее время оборота,**

умноженное на некоторый коэффициент. Практика показывает, что значение этого коэффициента должно превышать 2. В сетях с большим разбросом времени оборота при выборе тайм-аута учитывается и дисперсия этой величины.

Управление окном приема

Размер окна приема связан с наличием в данный момент места в буфере данных у принимающей стороны. Поэтому в общем случае окна приема на разных концах соединения имеют разный размер. Например, можно ожидать, что сервер, вероятно обладающий большим буфером, pošлет клиентской станции окно приема больше, чем клиент серверу. В зависимости от состояния сети то одна, то другая сторона могут объявлять новые значения окон приема, динамически уменьшая и увеличивая их.

Варьируя величину окна, можно влиять на загрузку сети. Чем больше окно, тем большая порция неподтвержденных данных может быть послана в сеть. Но если пришло большее количество данных, чем может быть принято модулем ТСП данные будут отброшены. Это приведет к излишним пересылкам информации и ненужному увеличению нагрузки на сеть и модуль ТСП.

С другой стороны, указание окна малого размера может ограничить передачу данных скоростью, которая определяется временем путешествия по сети каждого посылаемого сегмента. Чтобы избежать применения малых окон, в некоторых реализациях ТСП предлагается получателю данных откладывать реальное изменение размеров окна до тех пор, пока свободное место не составит 20-40 % от максимально возможного объема памяти для этого соединения. Но и отправителю не стоит спешить с посылкой данных, пока окно принимающей стороны не станет достаточно большим. Учитывая эти соображения, разработчики протокола ТСП предложили схему, согласно которой при установлении соединения заявляется большое окно, но впоследствии его размер существенно уменьшается. Существуют и другие прямо противоположные алгоритмы настройки окна, когда вначале выбирается минимальное окно, а затем, если сеть справляется с предложенной нагрузкой, его размер резко увеличивается.

Управлять размером окна приема может не только та сторона, которая посылает это окно, чтобы регулировать поток данных в свою сторону, но и вторая сторона — потенциальный отправитель данных. Если вторая сторона фиксирует ненадежную работу линии связи (регулярно запаздывают квитанции, часто требуется повторная передача), то она может по собственной инициативе уменьшить окно. В таких случаях действует правило: в качестве действующего размера окна выбирается минимальное из двух значений — значения, диктуемого приемной стороной, и значения, определяемого «на месте» отправителем.

Признаком перегрузки ТСП-соединения является возникновение очередей на промежуточных узлах (маршрутизаторах) и на конечных узлах (компьютерах). При переполнении приемного буфера конечного узла «перегруженный» протокол ТСП, отправляя квитанцию, помещает в нее новый, уменьшенный размер окна. Если он совсем отказывается от приема, то в квитанции указывается **окно нулевого размера**.

Однако даже после этого приложение может послать сообщение на отказавшийся от приема порт. Для этого сообщение должно сопровождаться

указателем срочности. В такой ситуации порт обязан принять сегмент, даже если для этого придется вытеснить из буфера уже находящиеся там данные.

После приема квитанции с нулевым значением окна протокол-отправитель время от времени делает контрольные попытки продолжить обмен данными. Если протокол-приемник уже готов принимать информацию, то в ответ на контрольный запрос он посылает квитанцию с указанием ненулевого размера окна.

Как видно из нашего далеко не полного описания двух протоколов транспортного уровня стека TCP/IP, на один из них — TCP — возложена сложная и очень важная задача обеспечение надежной передачи данных через ненадежную сеть.

С другой стороны, функциональная простота протокола UDP обуславливает простоту алгоритма его работы, компактность и высокое быстродействие. Поэтому те приложения, в которых реализован собственный, достаточно надежный, механизм обмена сообщениями, основанный на установлении соединения, предпочитают для непосредственной передачи данных по сети использовать менее надежные, но более быстрые средства транспортировки, в качестве которых по отношению к протоколу TCP и выступает протокол UDP. Протокол UDP может быть использован и в том случае, когда хорошее качество линий связи обеспечивает достаточный уровень надежности и без применения дополнительных приемов наподобие установления логического соединения и квитирования передаваемых пакетов. Заметим также, что поскольку протокол TCP основан на логических соединениях, он, в отличие от протокола UDP, **не годится для широковещательной и групповой рассылки**